

EI1061. ET-PR-2-4

TEMA II. PARALELISMO A NIVEL DE DATOS.

Parte III: GPU

Objetivos del cuestionario

- Comprender el modelo de programación CUDA y la separación entre host y device.
 - Entender la organización del paralelismo en GPU (grid, thread blocks y threads).
 - Interiorizar el modelo de ejecución SIMT y su relación con SIMD y MIMD.
 - Conocer la arquitectura interna de una GPU y el funcionamiento de los Streaming Multiprocessors (SM).
 - Analizar el papel de los mecanismos de planificación de hilos y bloques en la GPU.
 - Comprender cómo se oculta la latencia de memoria mediante multithreading masivo.
 - Conocer la jerarquía de memoria en GPU y su impacto en el rendimiento.
 - Entender la importancia del acceso eficiente a memoria, incluyendo el concepto de memory coalescing.
 - Identificar los principales factores que afectan al rendimiento en GPU.
 - Relacionar la arquitectura GPU con otros modelos de paralelismo, como SIMD y procesadores vectoriales.
-

Revisa la documentación sobre GPU del aula virtual y responde las siguientes cuestiones.

Cuestiones

BLOQUE 1 — Modelo de programación CUDA

1. **¿Qué es CUDA?**
Explica su objetivo y por qué facilitó el uso de las GPU para computación general.
2. **Explica la diferencia entre el código que se ejecuta en el host y el código que se ejecuta en el device.**
Indica también el significado de `__host__`, `__device__` y `__global__`.
3. **¿Qué significa que CUDA siga un modelo SIMT?**
Explica la relación entre SIMT, multithreading, MIMD y SIMD.

BLOQUE 2 — Modelo de ejecución

4. Define y relaciona los conceptos de Grid, Thread Block y CUDA Thread.
5. Explica cómo se paraleliza en CUDA un bucle secuencial como DAXPY.
Indica el papel de `blockIdx`, `threadIdx` y `blockDim`.
6. ¿Por qué se dice que el modelo de programación CUDA parece MIMD, aunque para obtener buen rendimiento hay que pensar en términos SIMD?
7. ¿Por qué CUDA exige que los *Thread Blocks* puedan ejecutarse de forma independiente y en cualquier orden?
8. Relaciona un *CUDA Thread* con una instrucción SIMD. Describe qué es un warp (SIMD thread).

BLOQUE 4 — Arquitectura GPU

9. Describe la arquitectura de una GPU según el modelo CUDA.
¿Qué es un multithreaded SIMD Processor o Streaming Multiprocessor (SM)?
10. Explica qué son los SIMD lanes y cómo se ejecutan las instrucciones dentro de un SM.
¿Por qué una instrucción puede necesitar varios ciclos para completarse?

BLOQUE 5 — Planificación

11. Explica el papel del Thread Block Scheduler y del SIMD Thread Scheduler.
12. ¿Cómo se obtiene el paralelismo a nivel de ejecución en una GPU y cómo se utiliza el multithreading para ocultar la latencia de memoria?
13. En la arquitectura Pascal, ¿a cuántos *SIMD Lanes* físicos se mapea una instrucción de un *Warp* (32 hilos) y cuánto tarda en completarse? Explica
¿cómo logra Pascal mantener activos los 64 *lanes* totales de un SM simultáneamente?

BLOQUE 6 — Memoria

14. Describe la organización de los registros en una GPU y explica la relación entre número de registros por hilo y número de hilos activos.
15. Describe los diferentes niveles de memoria de la GPU e indica dónde se encuentran y quién puede acceder a ellos.
Incluye global memory, shared/local memory y private memory.
16. ¿Qué es el memory coalescing y por qué es importante para el rendimiento de una GPU?
17. ¿Por qué las GPUs necesitan una cantidad masiva de registros (hasta 65,536) y cómo se asignan a los hilos?

BLOQUE 7 — Rendimiento

18. ¿Qué factores afectan al rendimiento en GPU?